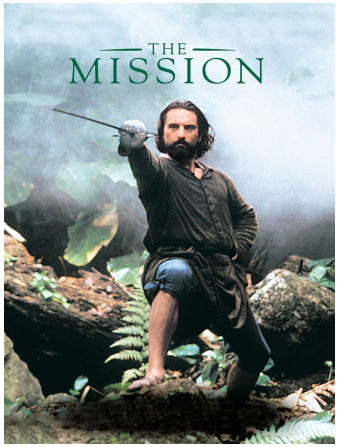
MACHINE LEARNING 2

Ciência de Dados para Negócios

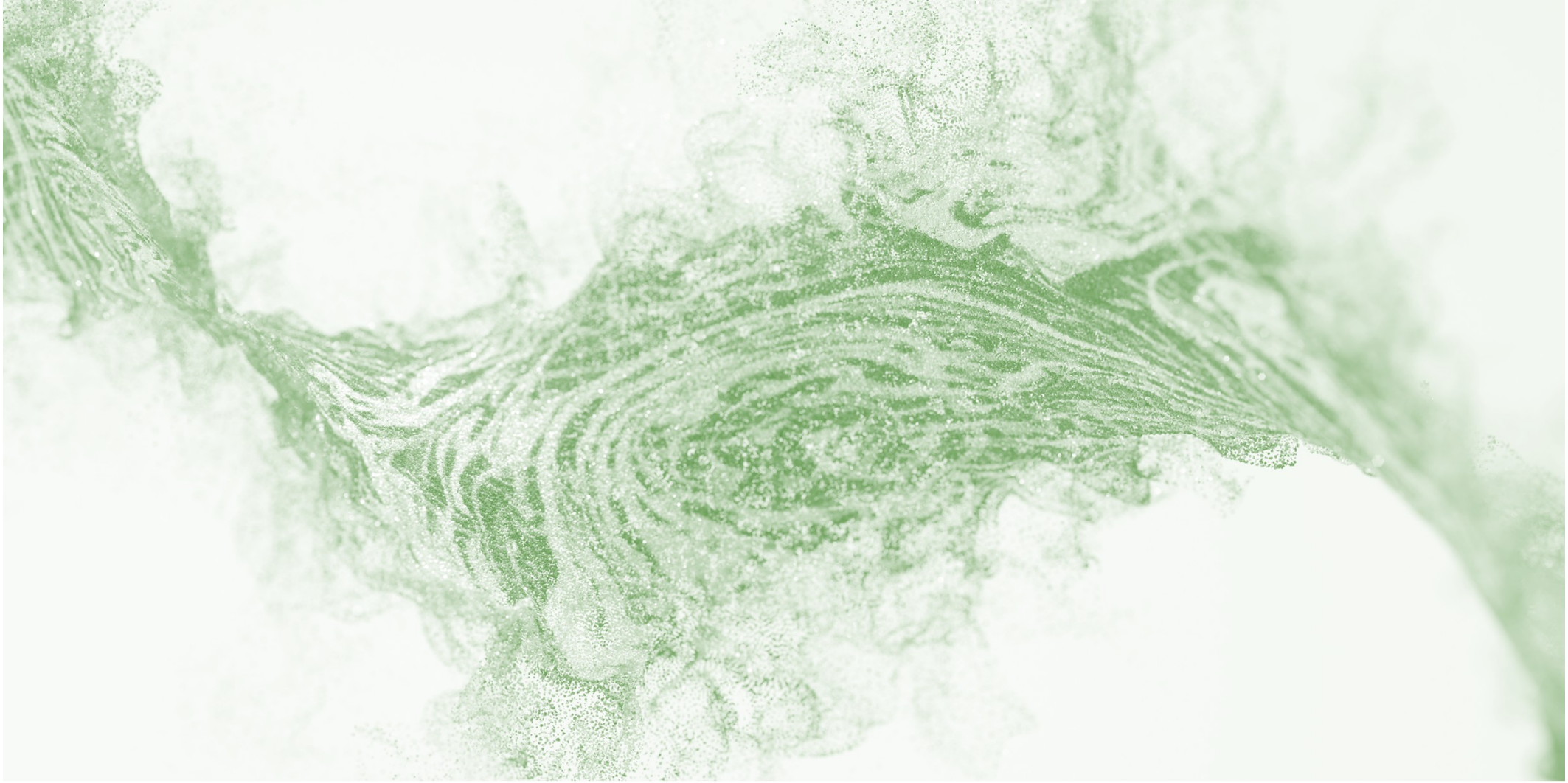
FATEC Votorantim



Prof. Luis Claudio

M. Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)



I – INTRODUÇÃO ÀS REDES NEURAIS ARTIFICIAIS

Prof. Luis Claudio

M.Sc. Eng. Elétrica(PUC-Rio-06)

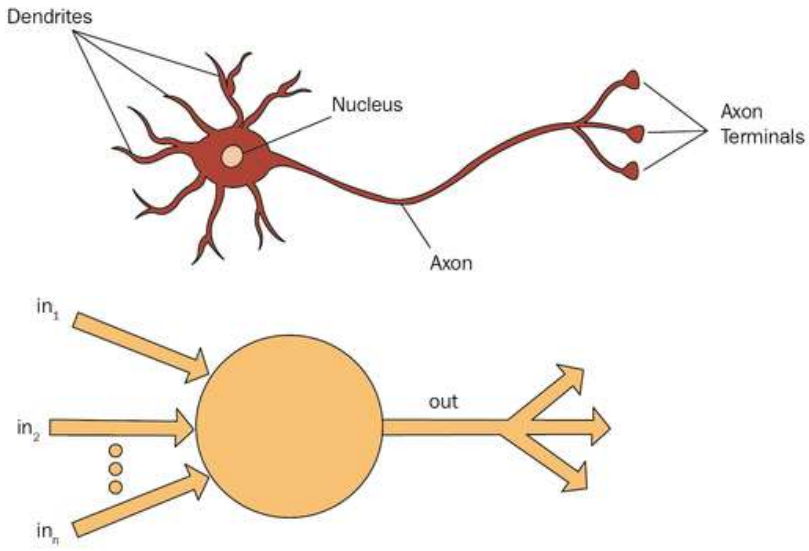
Eng. de Computação (ITA-99)

O QUE É UMA REDE NEURAL

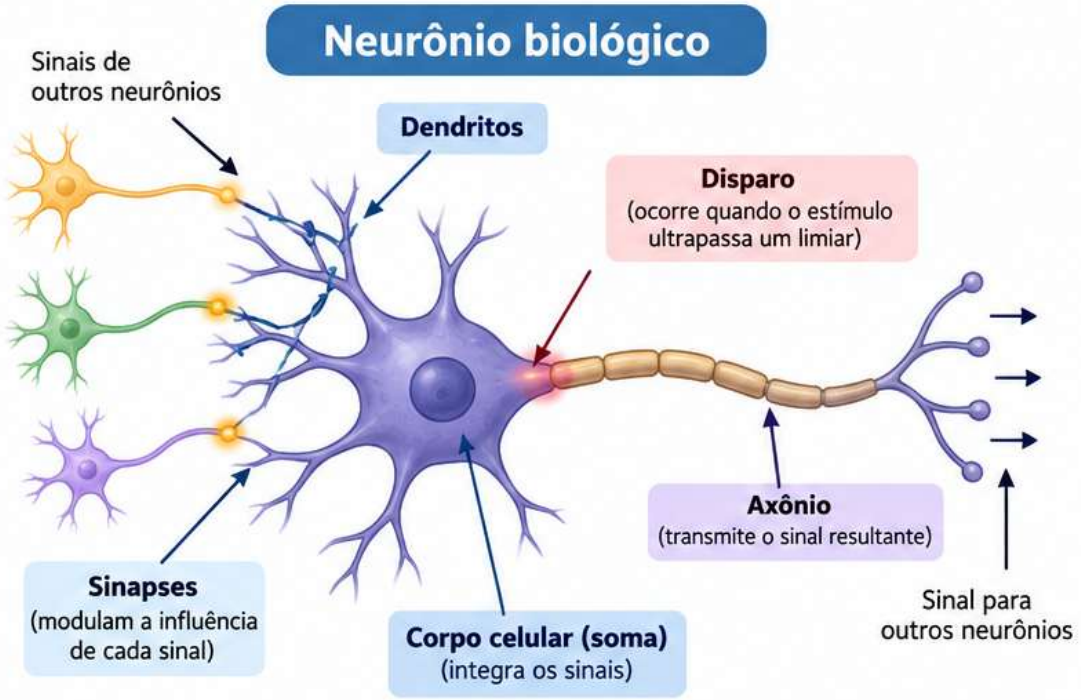
Na década de 1940, pesquisadores como **Warren McCulloch** e **Walter Pitts** tentaram responder à seguinte pergunta:

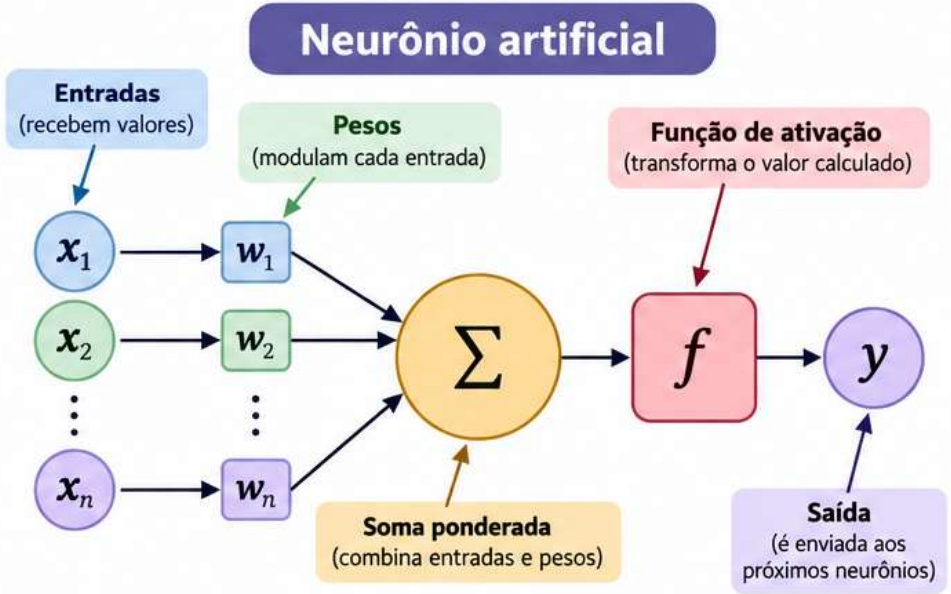
Como o cérebro consegue aprender e processar informações a partir de bilhões de neurônios interconectados?

A ideia foi criar um modelo matemático simplificado do neurônio biológico:



Neurônio biológico	Neurônio artificial
Dendritos recebem sinais de outros neurônios	**Entradas** $(x_1, x_2, \ldots, x_n)$ recebem valores
Sinapses modulam a influência de cada sinal	**Pesos** $(w_1, w_2, \ldots, w_n)$ modulam cada entrada
Corpo celular (soma) integra os sinais	**Soma ponderada** combina entradas e pesos
Disparo ocorre quando o estímulo ultrapassa um limiar	**Função de ativação** transforma o valor calculado
Axônio transmite o sinal resultante	**Saída** (y) é enviada aos próximos neurônios





$$z = \sum_{i=1}^{n} w_i x_i + b \quad \rightarrow \quad y = f(z)$$

NEURÔNIO ARTIFICIAL

O neurônio artificial é a **unidade básica de processamento** de uma rede neural. Ele recebe um conjunto de valores de entrada, realiza uma **combinação linear por meio de pesos** (*weights*) e de **um *bias***, e aplica uma **função de ativação** para produzir sua saída.

Matematicamente, o neurônio calcula a entrada líquida:

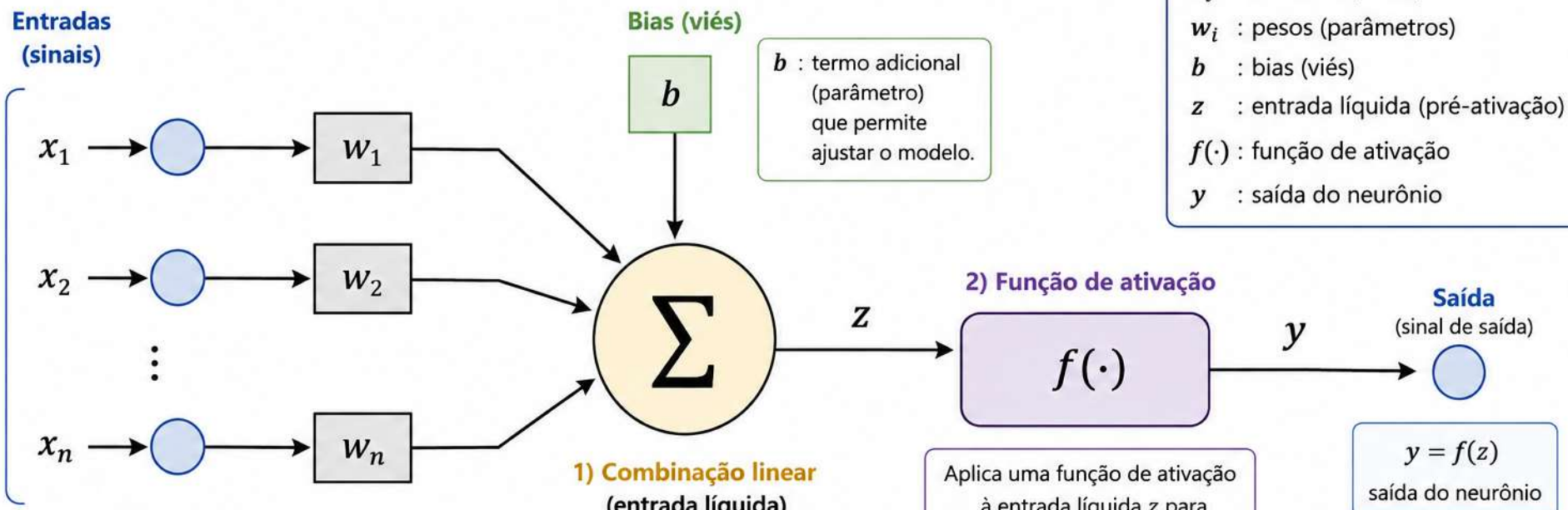
$$z = \sum_{i=1}^{n} w_i x_i + b$$

Em seguida, aplica uma função de ativação:

$$y = f(z)$$

A saída y é então encaminhada para os neurônios da camada seguinte **OU**, caso seja a última camada oculta, utilizada como saída final.

NEURÔNIO ARTIFICIAL



Legenda

- x_i : entradas (sinais)
- w_i : pesos (parâmetros)
- b : bias (viés)
- z : entrada líquida (pré-ativação)
- $f(\cdot)$: função de ativação
- y : saída do neurônio

b : termo adicional (parâmetro) que permite ajustar o modelo.

x_i : i-ésima entrada (sinal)

Pesos (parâmetros)

w_i : peso associado à entrada x_i

1) Combinação linear (entrada líquida)

$$z = \sum_{i=1}^{n} w_i x_i + b$$

z : entrada líquida (*net input*) ou pré-ativação

Aplica uma função de ativação à entrada líquida z para produzir a saída do neurônio.

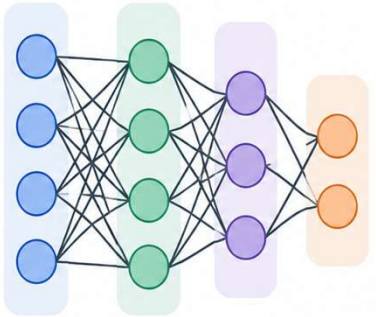
$y = f(z)$
saída do neurônio

Como funciona:

1. Cada entrada x_i é multiplicada pelo seu peso w_i.
2. Os valores ponderados são somados e o bias b é adicionado, produzindo z.
3. A função de ativação $f(\cdot)$ transforma z na saída y.
4. A saída y é enviada para os neurônios da próxima camada ou usada como saída final da rede.

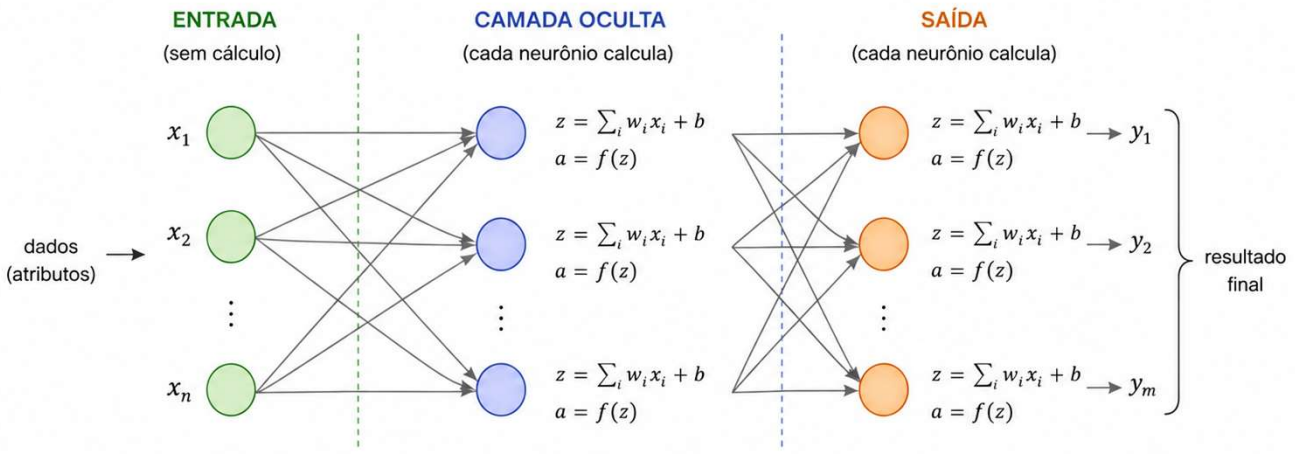
Observação histórica: Em modelos inspirados no neurônio biológico, o valor z já foi chamado de "potencial de ativação". Hoje, os termos mais comuns são entrada líquida, pré-ativação ou simplesmente z.

AS CAMADAS



Redes neurais são arquiteturas de **aprendizagem de máquina** compostas por **neurônios artificiais organizados em camadas,** capazes de aprender representações e padrões a partir dos dados.

Em uma rede *feedforward* convencional, distinguimos três tipos de camadas: **Entrada (Input),** uma ou mais **Camadas Ocultas (Hidden)** e **Saída (Output).**



ENTRADA

- Recebe os dados.
- Não faz cálculo.
- Não possui pesos nem *bias*.

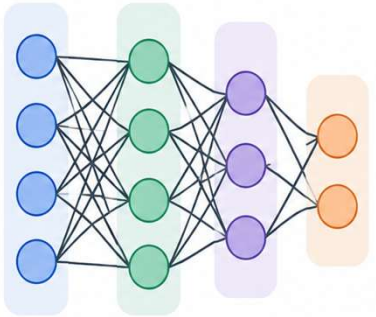
CAMADAS OCULTAS

- Cada neurônio calcula $z = \sum_i w_i x_i + b$.
- Aplica função de ativação: $a = f(z)$.
- Possui pesos (w) e *bias* (b).

SAÍDA

- Cada neurônio calcula $z = \sum_i w_i x_i + b$.
- Aplica (ou não) função de ativação: $a = f(z)$.
- Possui pesos (w) e *bias* (b).

A CAMADA DE ENTRADA



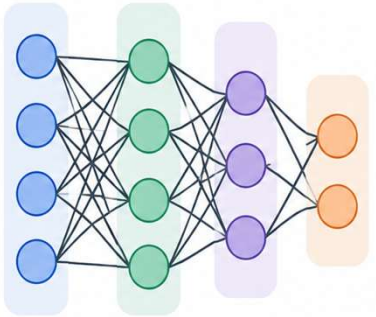
A camada de **Entrada** (*input layers*) recebe os dados de entrada (atributos), representados de **forma numérica** (dados tabulares, pixels de uma imagem etc.).

Em uma MLP tradicional (*), a camada de entrada apenas recebe os dados e os encaminha para a primeira camada oculta. Ela **não realiza processamento matemático** (não possui pesos, bias ou função de ativação).

Por esse motivo, alguns autores **não contabilizam a camada de entrada** ao medir a **profundidade** (*depth*) da rede neural.

* Em arquiteturas mais avançadas, a camada de entrada pode ser seguida por transformações específicas, realizadas por outras camadas, como uma camada de *embedding* em problemas de PLN.

AS CAMADAS OCULTAS



Nas camadas **Ocultas** (*hidden layers*), cada neurônio recebe as saídas da camada anterior, calcula uma combinação linear utilizando **pesos** e **bias** ($z = \sum_{i=1}^{n} w_i x_i + b$) e aplica uma **função de ativação** para produzir sua saída.

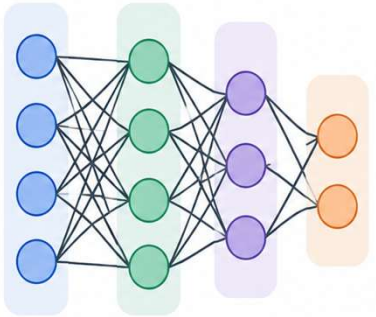
A **quantidade de camadas ocultas** e o **número de neurônios em cada camada** são **hiperparâmetros** definidos durante o projeto da rede.

Não existe uma regra universal para definir a quantidade de neurônios. Algumas **heurísticas** podem ser utilizadas como ponto de partida, como considerar valores próximos à quantidade de nós da camada de entrada ou potências de 2 (16, 32, 64, 128, 256).

As camadas ocultas quase **sempre utilizam funções de ativação**, que introduzem **não linearidade** e permitem que a rede aprenda **representações e relações complexas** a partir dos dados.

Sem funções de ativação nas camadas ocultas, sucessivas transformações lineares equivalem a uma única transformação linear, **limitando a capacidade da rede de modelar relações complexas.**

A CAMADA DE SAÍDA



Na camada de **Saída** (*output layer*) os neurônios também calculam uma combinação linear utilizando **pesos** e **bias** ($z = \sum_{i=1}^{n} w_i x_i + b$). Ou seja, ela **também tem pesos e bias aprendidos durante o treinamento**.

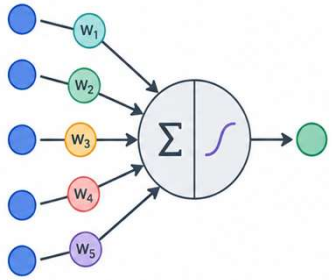
Quando aplicável, a camada de saída também usa **funções de ativação** para o processamento na camada de saída transforma as **representações internas da rede na resposta final do modelo**, adequada ao problema.

A **função de ativação** dependem do tipo de problema, como regressão (identidade), classificação binária (sigmoide) ou multiclasse (softmax).

Em arquiteturas tradicionais*, a **camada de saída é única**, embora possa conter **um ou vários neurônios** também conforme a tarefa.

* Em arquiteturas mais avançadas, a rede pode possuir **múltiplas cabeças de saída (*multiple output heads*)**, geralmente compartilhando uma estrutura comum (*backbone*).

NEURÔNIO ARTIFICIAL



Entradas (x) – Conjunto de valores fornecidos ao neurônio, representando os atributos ou características utilizados no processamento.

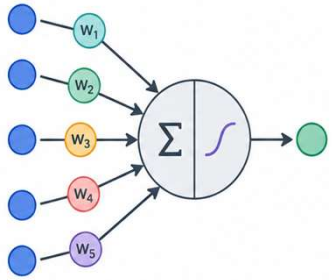
Pesos (w) – Parâmetros aprendidos durante o treinamento que determinam a influência de cada entrada na saída do neurônio.

Bias (b) – Termo adicional aprendido durante o treinamento que ajusta a combinação linear, proporcionando maior flexibilidade ao modelo.

Entrada Líquida ou Pré-Ativação (z) – Resultado da combinação linear entre os valores recebidos da camada anterior, seus respectivos pesos e o *bias*.

$$z = \sum_{i=1}^{n} w_i x_i + b = w_1 x_1 + w_2 x_2 + w_3 x_3 + \cdots + w_n x_n + b$$

NEURÔNIO ARTIFICIAL



Função de Ativação (f) – Função matemática aplicada à entrada líquida (z) para produzir a saída do neurônio.

- É aplicada sobre a entrada líquida (z).
- Introduz **não linearidade** no modelo, permitindo representar relações complexas entre as variáveis.
- O tipo de função de ativação depende da camada e do problema que se deseja resolver.

Saída (y) **ou Ativação** (a) – Saída produzida pelo neurônio a partir da entrada líquida (z), após a aplicação da função de ativação, quando utilizada.

- Nas camadas ocultas, constitui uma representação intermediária enviada à próxima camada; **na camada de saída**, corresponde à **resposta produzida pelo modelo**.

$$a = f(z)$$

Neurônio Artificial aplicado a dados tabulares

1 Dados tabulares

	idade	renda	score	...	alvo (y)
1	25	3000	650	...	0
2	40	8000	700	...	1
3	35	5000	620	...	0
⋮	⋮	⋮	⋮	⋮	⋮
n	28	4500	680	...	1

Cada linha = uma instância (exemplo)
Cada coluna (exceto alvo) = uma característica

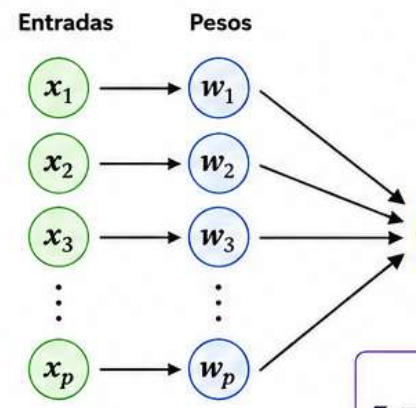
2 Após transformações / padronização

x_1	x_2	x_3	...	x_p	y
−0,45	−1,10	0,20	...	0,35	0
0,85	1,05	1,10	...	−0,15	1
0,30	−0,10	−0,30	...	0,60	0
⋮	⋮	⋮	⋮	⋮	⋮
−0,20	−0,60	0,40	...	0,10	1

Cada linha vira um vetor de **entrada**

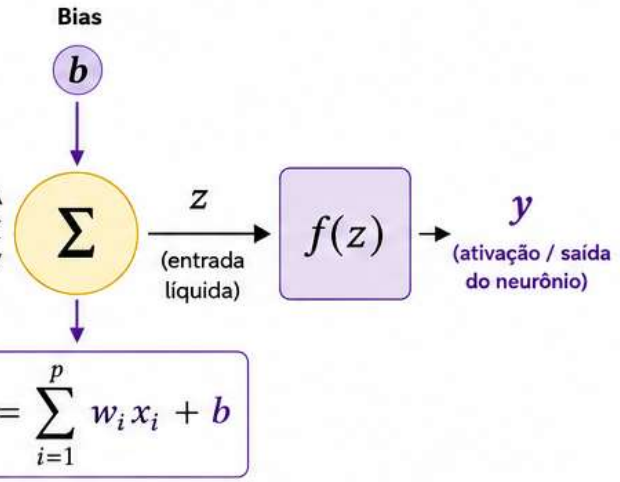
$$x = [x_1, x_2, \ldots, x_p]$$

3 Entradas do neurônio (uma instância)



Cada característica x_i é multiplicada pelo seu peso w_i

4 Entrada líquida (pré-ativação)



$$z = \sum_{i=1}^{p} w_i x_i + b$$

Soma de todos os produtos mais o *bias* (termo de ajuste)

A função de ativação f transforma z na **saída** y

5 Saída: o que queremos prever

y é a previsão gerada pelo neurônio.

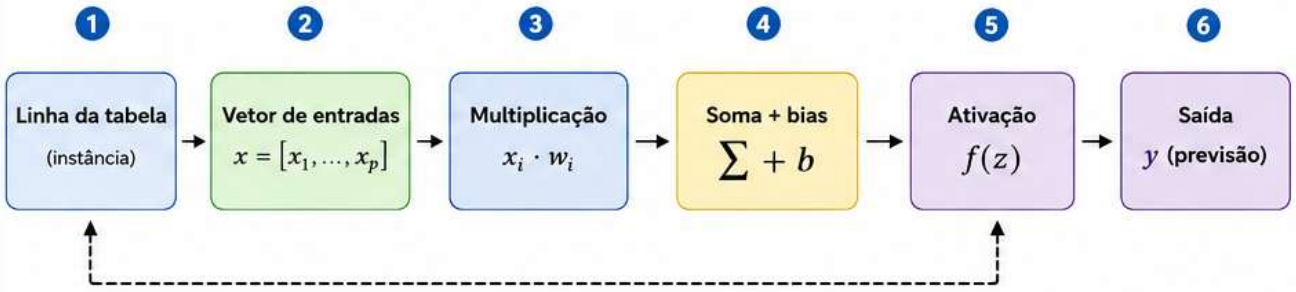
Comparada com o alvo (y real) para calcular o **erro** e **ajustar** os pesos e o *bias*.

$\hat{y}$		y (real)
0,12		0
0,87		1
0,23	Comparação (erro/loss)	0
⋮		⋮
0,75		0

Onde cada elemento atua:

- Pesos (w_i): aprendem a importância de cada característica.
- Bias (b): desloca a função, permitindo ajustar o ponto de decisão.
- Função de ativação (f): introduz não linearidade e define o tipo de saída.

Resumo do fluxo de uma instância (uma linha de dados)

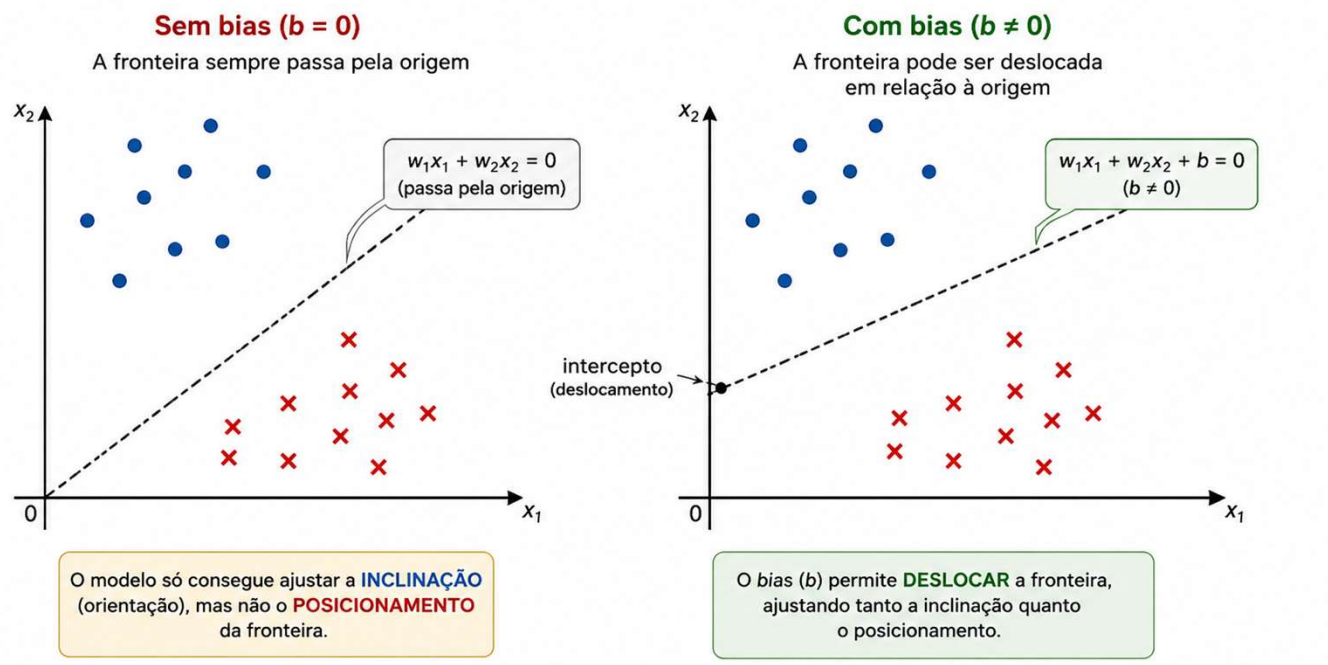


Ajuste dos pesos $(w_1, \ldots, w_p)$ e do *bias* (b) durante o treinamento para reduzir o erro em todas as instâncias

BIAS

O **bias (b)** permite **deslocar a fronteira de decisão** no espaço de entrada em relação à origem.

Sem o bias (**b = 0**), uma fronteira linear definida por $w_1x_1 + w_2x_2 = 0$ necessariamente passa pela origem. **Com bias** (**b ≠ 0**), a expressão $w_1x_1 + w_2x_2 + b = 0$ permite deslocar essa fronteira.



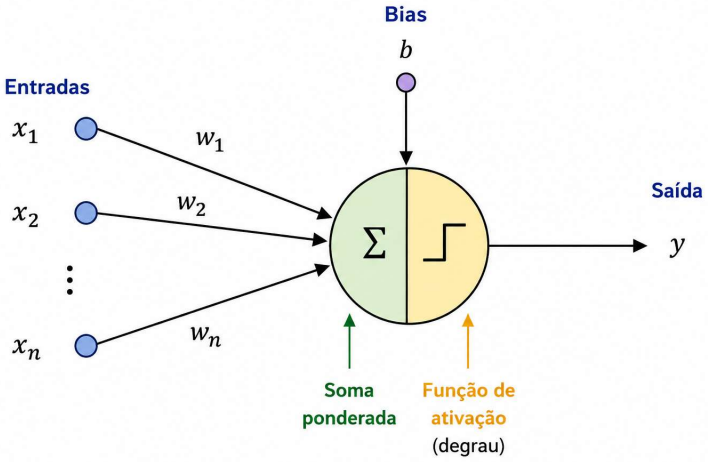


NEURÔNIO ARTIFICIAL
PERCEPTRON CLÁSSICO DE ROSENBLATT

Perceptron foi proposto por **Frank Rosenblatt em 1958** e é considerado um dos primeiros modelos de neurônio artificial com **capacidade de aprendizado**. Em sua forma mais simples, pode ser representado como uma **rede neural baseada em um único neurônio**.

O Perceptron tornou-se um dos modelos mais importantes na história das Redes Neurias por introduzir o **aprendizado de máquinas.**

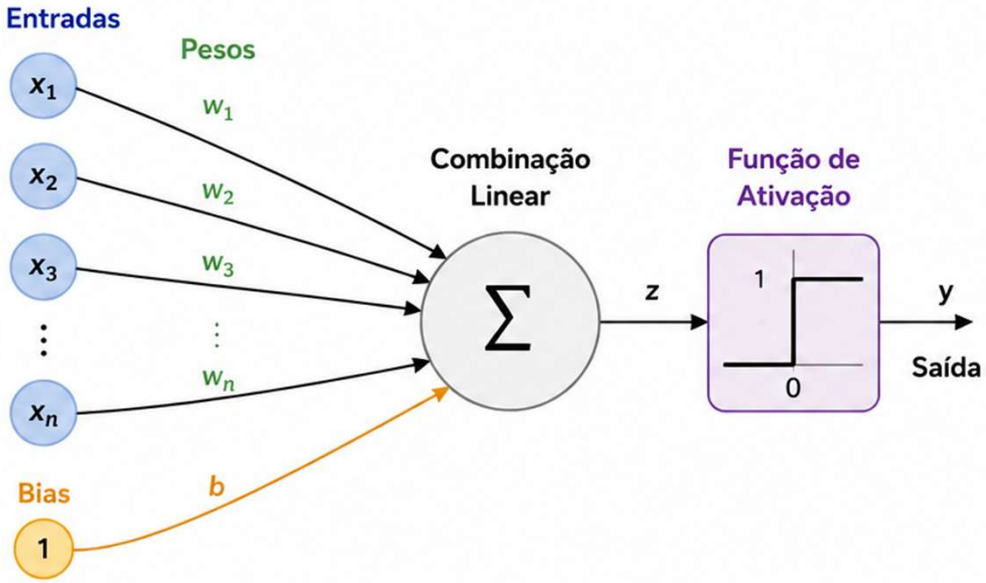
O **Perceptron** simples foi utilizado em problemas de **classificação binária** (**0 e 1, sim e não** ou **verdadeiro e falso**). Ele utilizava uma **função de ativação degrau** (*step* ou *threshold*) e aprendia uma **fronteira de decisão linear,** sendo capaz de resolver problemas **linearmente separáveis.**





Perceptron Clássico

Modelo mais simples de rede neural artificial para classificação linear binária



Combinação Linear:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

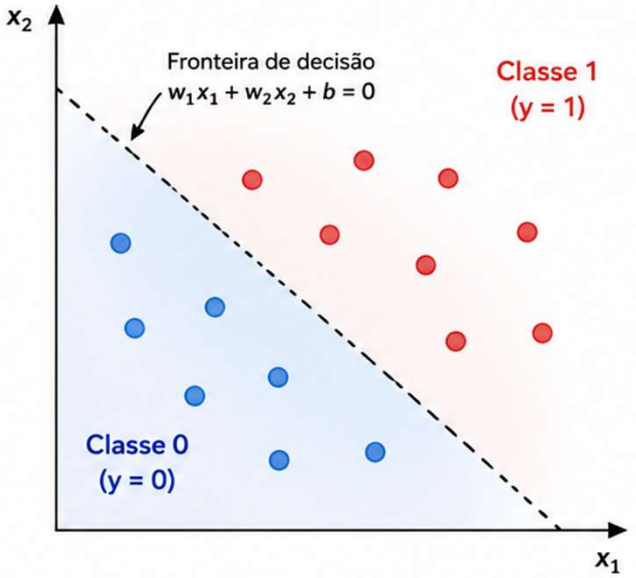
Função de Ativação (degrau):

$$y = f(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}$$

Onde:

- x_i = entradas (atributos)
- w_i = pesos
- b = bias (termo de viés)
- f = função de ativação (degrau)
- y = saída (classe prevista)

Separação Linear no Espaço de Entrada (2D)



Interpretação da Saída

- $y = 1$ → instância pertence à Classe 1
- $y = 0$ → instância pertence à Classe 0

Resumo: O perceptron aprende os pesos (w_1, w_2, ..., w_n) e o bias (b) a partir dos dados, de forma que a fronteira de decisão (linear) melhor separe as duas classes.



NEURÔNIO ARTIFICIAL
LIMITAÇÃO DO PERCEPTRON CLÁSSICO

O perceptron realiza classificação linear, isto é, busca aprender uma fronteira de decisão capaz de separar duas classes por meio de uma reta (2D), plano (3D) ou hiperplano (múltiplas dimensões).

É linear devido a uma **combinação linear dos atributos de entrada**. Os pesos (**w**) controlam a influência de cada atributo na decisão do modelo e o bias (**b**) desloca a fronteira de decisão no espaço de entrada. Normal...

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

Porém, se o problema for não linear...

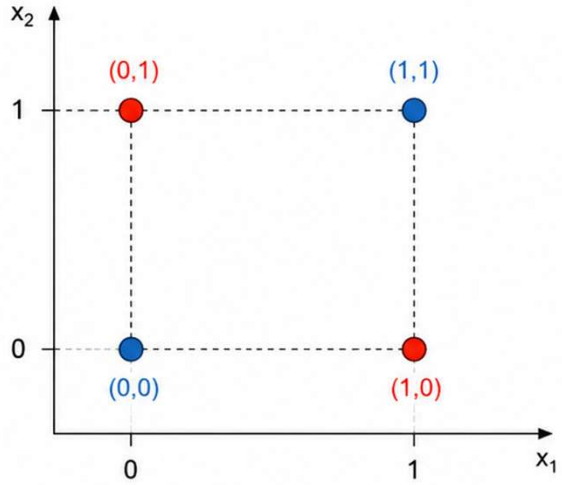
O **XOR** virou historicamente um exemplo clássico da limitação do perceptron linear. Não há fronteira que resolva o XOR!



NEURÔNIO ARTIFICIAL
PROBLEMA NÃO LINEAR MAIS SIMPLES: XOR

Problema XOR

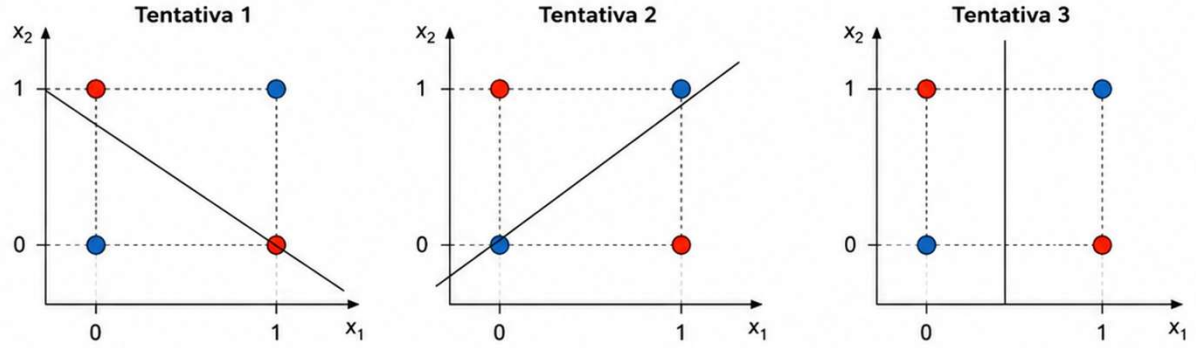
x_1	x_2	x_1 XOR x_2 (saída)
0	0	0
0	1	1
1	0	1
1	1	0



Classe 0 (saída = 0)

Classe 1 (saída = 1)

Por que o perceptron (classificação linear) não resolve?



Nenhuma reta (ou hiperplano) consegue separar as classes corretamente.

HIPERPARÂMETROS

Parâmetros

São **aprendidos** pela rede durante o treinamento. Representam **o que** o modelo aprende a partir dos dados.

- Exemplos: **pesos (weights)** e **vieses (biases)**.

Hiperparâmetros

São **definidos pelo cientista de dados**.

Determinam **como** e em **quais condições** o aprendizado ocorrerá.

- Exemplos: número de camadas (“profundidade”), número de neurônios por camada (“largura”), função de ativação, tamanho de lote (*batch size*), número de épocas, função de perda (*loss function*), otimizador e taxa de aprendizado (*learning rate*) entre outras.

HIPERPARÂMETROS
ARQUITETURA VERSUS TREINAMENTO

Embora não seja uma divisão canônica, eu gosto de dividir em:

Hiperparâmetros de Arquitetura

São paramentos que definem **como a rede neural é estruturada**:

- Número de camadas ocultas
- Número de neurônios por camada oculta
- Função de ativação

Hiperparâmetros de Treinamento/Otimização

São parâmetros que definem **como o aprendizado acontecerá**:

- Tamanho do lote (*batch size*)
- Número de épocas (*epochs*)
- Função de perda (*loss function*)
- Otimizador
- Taxa de aprendizado (*learning rate*)


ReLU / tanh /
sigmoid
HIPERPARÂMETROS
DE ARQUITETURA
(estrutura da rede)



HIPERPARÂMETROS
QUANTIDADE DE CAMADAS OCULTAS

Não existe uma regra que determine o número ideal de camadas ocultas.

A escolha inicial depende principalmente da **complexidade do problema** e deve ser posteriormente **avaliada experimentalmente**.

Em geral, quanto **mais camadas, maior a capacidade** da rede de construir representações mais complexas, mas também **maior o custo computacional e a complexidade do treinamento**.

Na pratica...

- Para problemas tabulares tradicionais, **1 a 3 camadas ocultas** costumam ser um ponto de partida razoável.
- Redes com dezenas ou centenas de camadas estão normalmente associadas a arquiteturas e problemas muito mais complexos (LLMs, por exemplo).



HIPERPARÂMETROS
QUANTIDADE DE NEURÔNIOS POR CAMADA

Diferentemente das camadas de entrada e saída, cuja quantidade de neurônios está diretamente relacionada, respectivamente, à dimensionalidade dos dados e ao tipo de problema, **não existe uma regra que determine a quantidade ideal de neurônios nas camadas ocultas.**

Aqui também, a escolha inicial é uma **hipótese de arquitetura**, posteriormente validada experimentalmente.

Na prática...

- É comum utilizar valores como **32, 64, 128, 256,** por convenção e também por eficiência computacional (legado).
- A arquitetura pode manter, aumentar ou reduzir a quantidade de neurônios entre as camadas. Um **afunilamento progressivo** é uma estratégia bastante utilizada (mas existem outras, como "V" etc.).
- Mais neurônios aumentam a **capacidade de representação** da rede, mas também aumentam o **número de parâmetros e o custo computacional.**



HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO

As funções de ativação são aplicadas à **Entrada Líquida ou Pré-Ativação (z)** calculada por cada neurônio.

Nas **camadas ocultas**, sua principal função é **introduzir não linearidade**, permitindo que a rede aprenda relações e fronteiras de decisão complexas, que não poderiam ser representadas por sucessivas transformações lineares.

Entre as principais funções de ativação estão:

- Sigmoid
- Tanh
- ReLU

Há diversas outras, como **Leaky ReLU, ELU, GELU, SiLU/Swish e Softplus.**



FUNÇÃO DE ATIVAÇÃO
TANGENTE HIPERBÓLICA

A função sigmoide recebe qualquer valor de **z entre -∞ e +∞** e produz uma saída entre **0 e 1**. É uma função **não linear**, contínua e com formato de "S" que produz uma **limitação/compressão** da faixa das ativações.

É **centrada em 0,5** (para **z = 0**, a saída é **0,5**); para **z ≈ 2,2**, a saída é aproximadamente **0,9**; para **z ≈ -2,2**, aproximadamente **0,1**.

$$f(z) = \frac{1}{1 + e^{-z}}$$

Para valores elevados de |z|, a função entra em uma região de **saturação**, na qual grandes alterações em z produzem pequenas alterações na saída. Essa saturação pode provocar o problema de ***vanishing gradiente*** (gradiente que desaparece), dificultando o treinamento de redes mais profundas*.

* Na retropropagação, para calcular o **gradiente da função de perda em relação aos pesos**, usamos a **regra da cadeia**, que envolve as **derivadas das funções de ativação**. Por isso a saturação é problemática: uma curva quase horizontal gera derivada ≈ 0 e o gradiente tende a zero, prejudicando a atualização dos pesos (em outras palavras "a rede para de aprender").

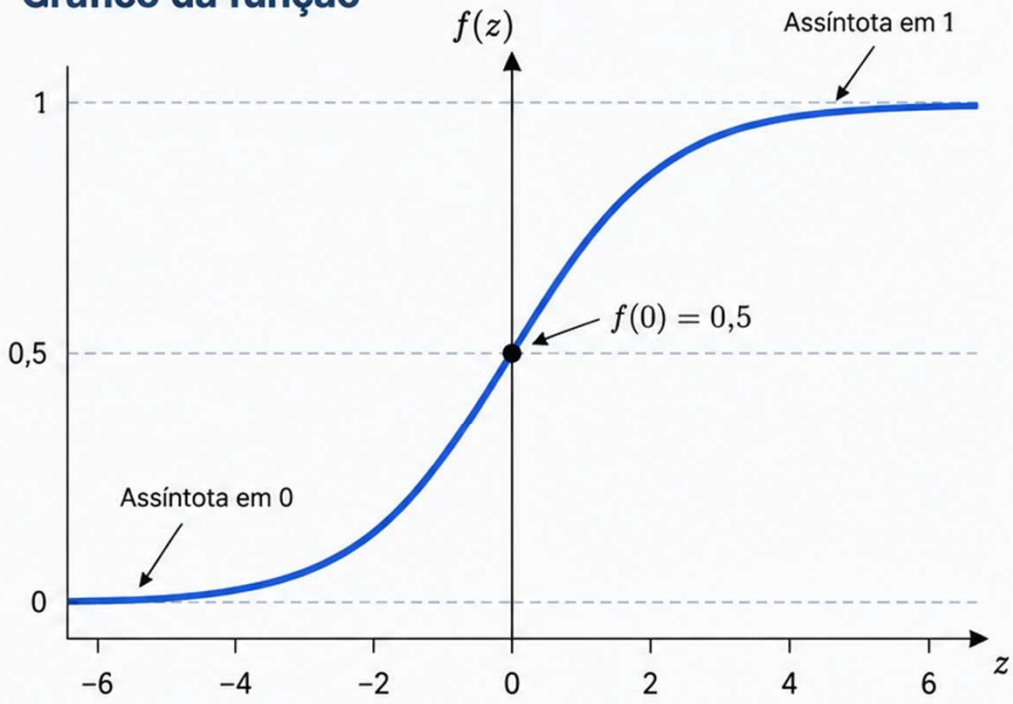
Função Sigmoide

$$f(z) = \frac{1}{1 + e^{-z}}$$

Principais características

- Recebe qualquer valor de z $(\text{de } -\infty \text{ a } +\infty)$ e produz uma saída entre 0 e 1.
- É uma função não linear, contínua e com formato de “S”.
- Para $z = 0$, a saída é $0{,}5$.
- Para valores elevados de $|z|$, a função entra em uma região de saturação.
- A saturação pode provocar o problema de vanishing gradient, dificultando o treinamento de redes mais profundas.
- Atualmente, é pouco utilizada em camadas ocultas, sendo amplamente substituída pela ReLU e suas variantes.

Gráfico da função



Exemplos de valores

z	−4	−2	0	2	4
$f(z)$	0,018	0,119	0,500	0,881	0,982



FUNÇÃO DE ATIVAÇÃO
TANGENTE HIPERBÓLICA

A função tanh recebe qualquer valor de **z entre -∞ e +∞** e produz uma saída entre **-1 e 1**. Também é **não linear**, contínua e com formato de "S" e que também produz uma **limitação/compressão** da faixa das ativações.

É **centrada em 0** (para **z = 0**, a saída é **0**); para **z ≈ 1,47**, a saída é aproximadamente **0,9**; para **z ≈ -1,47**, aproximadamente **-0,9**.

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Para valores elevados de |z|, também apresenta **saturação**, aproximando-se de -1 ou 1. Ou seja, também está sujeita ao ***vanishing gradient***.

Uma vantagem da tanh em relação à sigmoid é produzir saídas centradas em zero, **preservando valores negativos e positivos**.

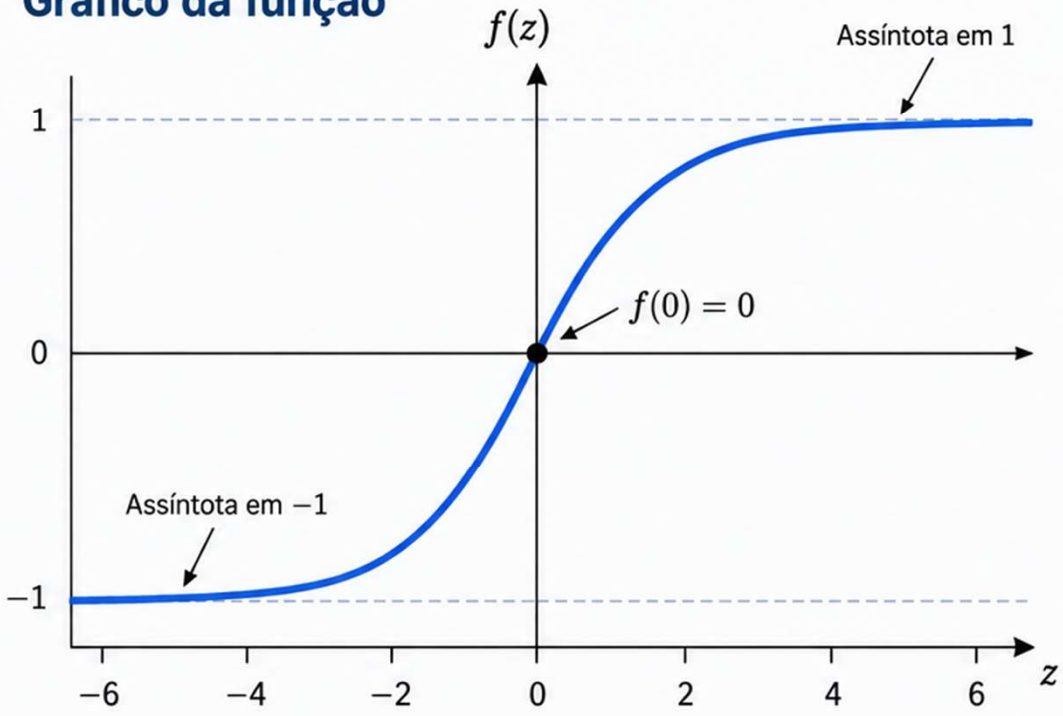
Função Tangente Hiperbólica (*Tanh*)

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Principais características

- Recebe qualquer valor de z (de $-\infty$ a $+\infty$) e produz uma saída entre -1 e 1.
- É uma função não linear, contínua e com formato de "S".
- É simétrica em relação à origem: $tanh(-z) = -\tanh(z)$.
- Para $z = 0$, a saída é 0.
- Para valores elevados de $|z|$, a função entra em uma região de saturação (próxima de -1 ou 1).
- Também pode sofrer com o problema de vanishing gradient, embora em menor grau que a função sigmoide.
- Foi amplamente utilizada em redes neurais, sendo hoje menos comum devido ao melhor desempenho da ReLU e suas variantes.

Gráfico da função



Exemplos de valores

z	−4	−2	0	2	4
$tanh(z)$	−0,999	−0,964	0,000	0,964	0,999



HIPERPARÂMETROS
FUNÇÃO RELU

ReLU (*Rectified Linear Unit*) é atualmente uma das funções de ativação mais utilizadas em **camadas ocultas**.

Seu comportamento é **extremamente simples**:

- Para **z ≤ 0**, a saída é **0**;
- Para **z > 0**, a saída é o **próprio z**.

$$f(z) = \max(0, z)$$

Apesar de sua simplicidade, é **não linear**, pois apresenta uma mudança de comportamento em **z = 0**.

- Em **z=0**, a derivada **não é matematicamente definida**; nas implementações computacionais, adota-se convencionalmente um valor para permitir o treinamento.
- Para **z > 0**, sua derivada é **1**, evitando a saturação observada na sigmoide. Isso contribui para reduzir o problema de ***vanishing gradient*** e facilita o treinamento de redes profundas.

* Como limitações, a ReLU **não limita superiormente as ativações**, que podem assumir valores muito elevados e contribuir para instabilidades no treinamento. Além disso, para z<0, sua derivada é 0, podendo ocorrer o problema conhecido como **dying ReLU**. Variantes como **Leaky ReLU** introduzem uma pequena inclinação para valores negativos, buscando reduzir este último problema.

ReLU — Rectified Linear Unit

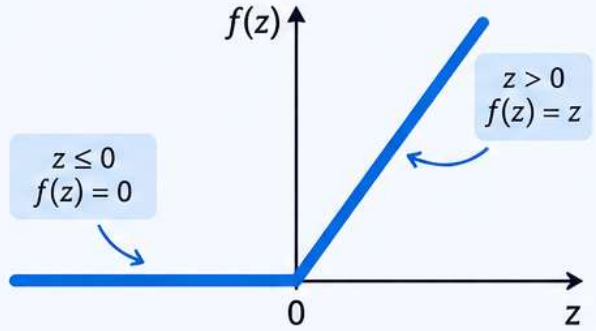
(Unidade Linear Retificada)

- Uma das funções de ativação mais utilizadas em **camadas ocultas.**
- Simples e eficiente.
- É **não linear**, pois muda o comportamento em $z = 0$.
- Para $z > 0$, sua derivada é 1, evitando a saturação e ajudando a reduzir o problema de **vanishing gradient.**

Definição

$$f(z) = \max(0, z)$$

Gráfico



Exemplos de valores

z	−4	−2	0	2	4
$f(z)$	0	0	0	2	4
$f'(z)$	0	0	(indef.)	1	1

Obs.: em $z = 0$, a derivada não é definida. Em implementações práticas, costuma-se usar 0 ou 1.

Vantagens

- Não satura para $z > 0$ (derivada = 1)
- Ajuda a reduzir o vanishing gradient
- Computacionalmente simples e eficiente
- Funciona bem em redes profundas

Limitações

- Derivada = 0 para $z < 0$
- Pode causar o problema de "dying ReLU" (neurônios que param de ativar)

Em resumo: a ReLU é simples, eficiente e evita a saturação na região positiva, o que facilita o treinamento de redes profundas. Por isso, é hoje a função de ativação mais usada em camadas ocultas.



HIPERPARÂMETROS
FUNÇÃO RELU

A **ReLU tornou-se uma função padrão para camadas ocultas,** especialmente em redes profundas, porque é **simples, computacionalmente barata e reduz o problema** de *vanishing gradient* na região positiva.

$$f(z) = \max(0, z)$$

Mas hoje o cenário é mais diversificado:

- **ReLU e variantes** → muito comuns em CNNs e diversas redes *feedforward*.
- **GELU** → tornou-se muito comum em **Transformers.**
- **SiLU/Swish** → utilizada em arquiteturas modernas, inclusive redes de visão.

HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO NA SAÍDA

As funções de ativação são aplicadas à **Entrada Líquida ou Pré-Ativação (z)** calculada por cada neurônio.

Na **camada de saída**, sua principal função é **adequar a saída da rede ao tipo de problema que está sendo modelado e ao formato esperado da resposta**.

Dessa forma, a função de ativação pode determinar características da saída, como seu **intervalo de valores e sua interpretação**.

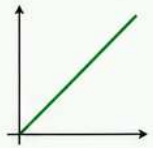
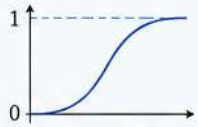
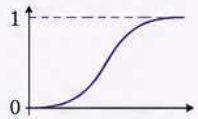
A escolha da função de ativação na camada de saída está, portanto, diretamente relacionada ao **tipo de problema**, devendo também ser **compatível com a função de perda** utilizada durante o treinamento.

HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO NA SAÍDA

A escolha da função de ativação e da quantidade de neurônios da **camada de saída** depende do tipo de problema:

- **Regressão:** geralmente **1 neurônio** com função **Identidade (Linear).** A saída pode assumir valores contínuos sem limitação de intervalo.
- **Classificação Binária:** geralmente **1 neurônio** com **Sigmoid.** A saída varia entre **0 e 1** e pode ser interpretada como a **probabilidade da classe positiva.**
- **Classificação Multiclasse:** geralmente **1 neurônio por classe** com **Softmax.** As saídas representam probabilidades associadas às classes e **somam 1.**
- **Classificação Multilabel:** geralmente **1 neurônio por classe** com **Sigmoid aplicada independentemente a cada neurônio.** Cada saída representa a probabilidade de uma classe, permitindo que **várias classes sejam atribuídas simultaneamente.**

Configuração da Camada de Saída em Redes Neurais

Tipo de Problema	Neurônios na Saída	Função de Ativação	Saída Produzida	Interpretação	Exemplo
Regressão Prever um ou mais valores contínuos (reais).	**1 (geralmente) ou mais** $\bigcirc \rightarrow y_1$ $\bigcirc \rightarrow y_2$ $\vdots \quad \vdots$ $\bigcirc \rightarrow y_k$ (1 neurônio para cada variável-alvo)	**Linear (identidade)**  $f(z) = z$	**Valores reais (sem restrição)** Ex.: $y \in (-\infty, +\infty)$	Cada neurônio produz um valor real que pode assumir qualquer valor (no intervalo real).	**Exemplos:** • Preço de um imóvel • Temperatura • LTV • Prever temperatura e umidade (2 saídas)
Classificação Binária Prever se pertence à classe positiva (1) ou negativa (0).	**1** $\bigcirc \rightarrow y$	**Sigmoid**  $f(z) = \frac{1}{1+e^{-z}}$	**Uma probabilidade entre 0 e 1** Ex.: $y \in [0, 1]$	Representa a probabilidade da classe positiva (1). Usa-se um limiar (ex.: 0,5) para decidir a classe.	**Exemplos:** • Spam / Não spam • Fraude / Não fraude • Cliente vai comprar / Não vai comprar
Classificação Multiclasse Existem várias classes possíveis, mas cada exemplo pertence apenas a uma delas.	**1 por classe (k classes)** $\bigcirc \rightarrow z_1$ $\bigcirc \rightarrow z_2$ $\vdots$ $\bigcirc \rightarrow z_k$	**Softmax** $f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$ (aplicada ao vetor z)	**Distribuição de probabilidades (soma = 1)** Ex.: $[0, 0.15, 0.70, 0.15]$ $(\sum_i p_i = 1)$	A classe prevista é aquela com a maior probabilidade. As probabilidades de todas as classes somam 1.	**Exemplos:** • Tipo de animal: gato, cachorro ou cavalo • Meio de transporte: carro, ônibus ou avião
Classificação Multirrótulo (Multi-label) Um exemplo pode pertencer a várias classes ao mesmo tempo.	**1 por classe (k classes)** $\bigcirc \rightarrow y_1$ $\bigcirc \rightarrow y_2$ $\vdots$ $\bigcirc \rightarrow y_k$	**Sigmoid** (em cada neurônio)  $f(z) = \frac{1}{1+e^{-z}}$	**Probabilidade independente para cada classe** Ex.: $[0.95, 0.82, 0.10, 0.67]$ (não somam 1)	Cada neurônio indica a probabilidade daquela classe estar presente, independentemente das demais.	**Exemplos:** • Em uma imagem pode haver: pessoa, carro, árvore e bicicleta simultaneamente • Doenças que um paciente pode ter

Regra prática: A configuração da camada de saída depende do tipo de problema. A quantidade de neurônios e a função de ativação são escolhidas de acordo com o formato da resposta que se deseja produzir.


Learning rate
Batch size
Épocas
Otimizador
Regularização
HIPERPARÂMETROS
DE TREINAMENTO
(aprendizado / otimização)



HIPERPARÂMETROS
TAMANHO DO LOTE (BATCH SIZE)

O **tamanho do lote (*batch size*)** determina **quantas instâncias do conjunto de treinamento são processadas antes de cada atualização dos pesos e biases**.

Durante uma época, o conjunto de treinamento é dividido em lotes. Por exemplo, para **10.000 instâncias** e ***batch_size*** = 100, uma época terá aproximadamente **100 atualizações dos parâmetros**.

É um **hiperparâmetro de treinamento** definido experimentalmente, pois **não existe uma regra universal** para determinar seu valor. Valores como **16, 32, 64, 128 e 256** são frequentes, mas *batches* de **512, 1024 ou maiores** também podem ser utilizados, especialmente em treinamentos de grande escala.

- **Batch grande:** observa muitas amostras antes de decidir → estima uma **direção mais estável** → porém precisa processar **muitas amostras antes de reavaliar a direção e atualizar os pesos novamente.**
- **Batch pequeno:** observa poucas amostras → estima uma **direção menos estável/mais ruidosa** → porém rapidamente pode **reavaliar a direção e fazer uma nova atualização.**



HIPERPARÂMETROS
NÚMERO DE ÉPOCAS (EPOCHS)

Uma **época** corresponde a **uma passagem completa de todo o conjunto de treinamento pela rede neural**. Quanto mais épocas, mas vezes o conjunto completo de treinamento é apresentado à rede.

Durante uma época, os dados são processados em lotes, e os parâmetros são atualizados à medida que esses lotes são processados.

Por exemplo, com **10.000 instâncias** e ***batch_size* de 100**, teremos **100 lotes por época**. Se usar como critério de parada a quantidade de épocas igual a **50**, serão realizadas **5.000 atualizações dos parâmetros**.

Obs.: Note que **poucas épocas** podem resultar em ***underfitting*** (aprendizado insuficiente) e, por outro lado, **muitas épocas** podem resultar em ***overfitting*** (além de aumentar o tempo de treinamento e custo de processamento).



HIPERPARÂMETROS
FUNÇÃO DE PERDA (LOSS FUNCTION)

A **função de perda** (***L***) é uma função matemática utilizada para **medir o erro do modelo**, comparando a saída prevista **y^** com o valor esperado **y**.

$$L(y, \hat{y})$$

Durante o treinamento, o objetivo é **minimizar o valor da função de perda** por meio do ajuste dos pesos e biases da rede.

A forma de calcular a perda depende do **tipo de problema**.

Por exemplo:

- **Regressão: MSE (*Mean Squared Error*)** — calcula a média dos erros ao quadrado, penalizando mais fortemente erros maiores; **MAE (*Mean Absolute Error*)** — calcula a média dos erros absolutos e é menos sensível a valores extremos.
- **Classificação Binária (Binary Cross-Entropy)** — penaliza o modelo de acordo com a **probabilidade atribuída à classe correta**, considerando duas classes; previsões confiantes e incorretas recebem perdas maiores.
- **Classificação Multiclasse (Categorical Cross-Entropy)** — penaliza o modelo de acordo com a **probabilidade atribuída à classe correta**, considerando múltiplas classes; previsões confiantes e incorretas recebem perdas maiores.



HIPERPARÂMETROS
TAXA DE APRENDIZADO

A **taxa de aprendizado** determina a **magnitude das alterações** realizadas nos pesos e biases a cada atualização durante o treinamento. Assim, controla o **tamanho do passo** dado na direção determinada pelo gradiente.

A taxa de aprendizado determina **quanto os parâmetros são alterados em cada atualização**, e não **quantas vezes** a atualização ocorre.

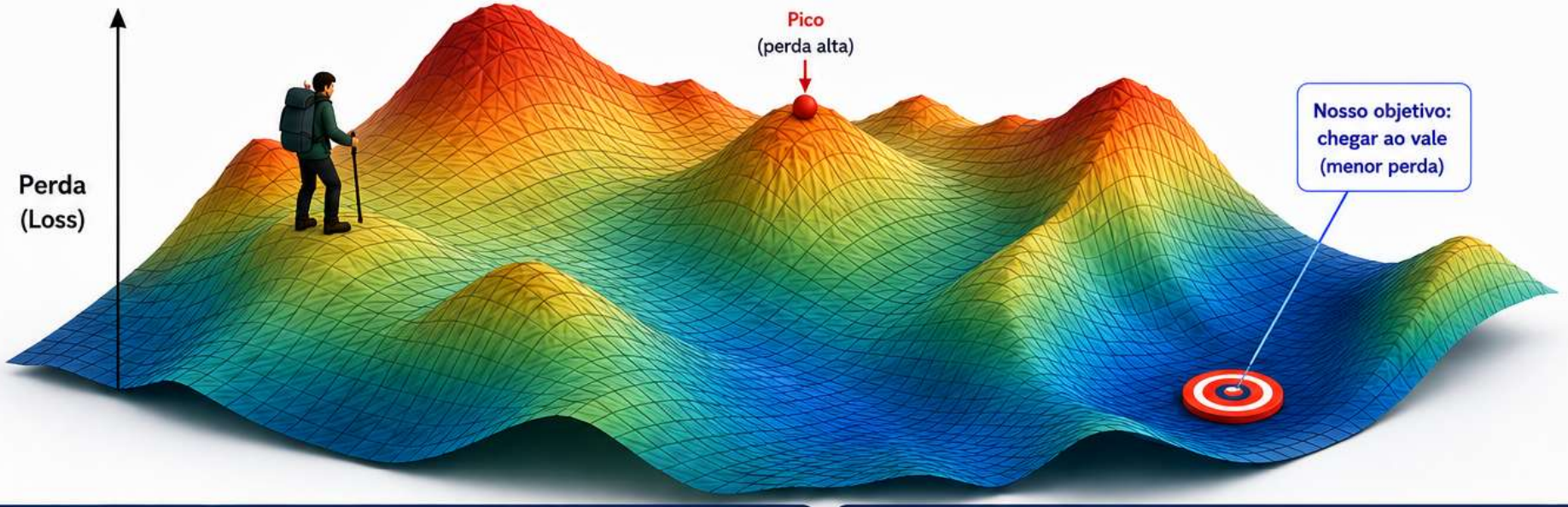
É normalmente representada por **η** (*eta*) ou **α**.

Valores típicos de ***learning rate*** são 0.01, 0.001 e 0.0001.

Obs.: Note que **taxas muito altas** podem dificultar a convergência, pois produzem **passos muito grandes**, podendo ultrapassar regiões de mínimo e, por outro lado, **taxas muito baixas** podem tornar o aprendizado **excessivamente lento**.

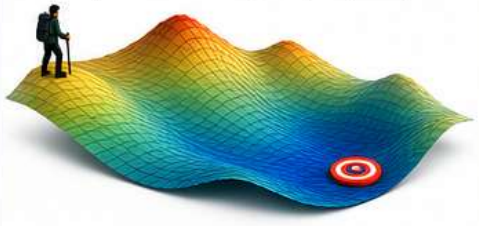
OTIMIZAÇÃO: BATCH SIZE E LEARNING RATE

Uma jornada para encontrar o vale (mínimo da função de perda)



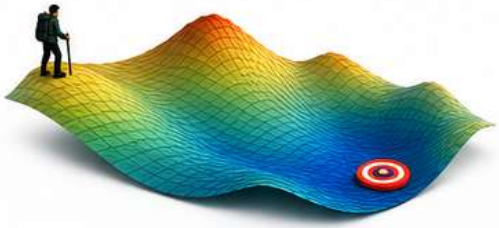
EFEITO DO BATCH SIZE (frequência e qualidade da direção)

BATCH PEQUENO
(muitas atualizações por época)



- ✓ Usa poucas amostras para calcular a direção (gradiente mais ruidoso).
- ✓ Direção menos precisa.
- ✓ Faz muitas atualizações (passos) por época.
- ✓ Corrige a rota com frequência.

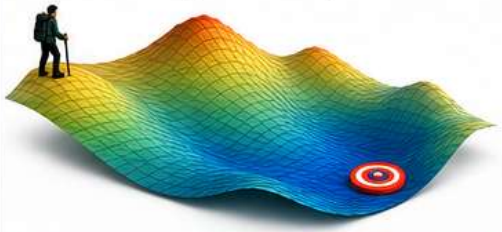
BATCH GRANDE
(poucas atualizações por época)



- ✓ Usa muitas amostras para calcular a direção (gradiente mais estável).
- ✓ Direção mais precisa.
- ✓ Faz poucas atualizações (passos) por época.
- ✓ Corrige a rota com menos frequência.

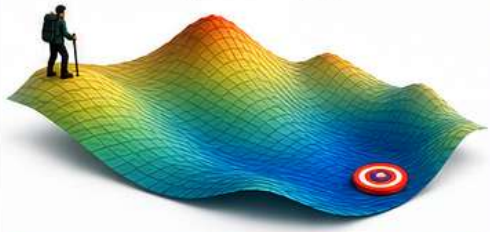
EFEITO DO LEARNING RATE (tamanho do passo)

LEARNING RATE PEQUENA
(passos curtos)



- ✓ Passos curtos a cada atualização.
- ✓ Anda pouco por atualização.
- ✓ Convergência mais lenta.
- ✓ Menor risco de ultrapassar o vale.

LEARNING RATE GRANDE
(passos longos)



- ✓ Passos longos a cada atualização.
- ✓ Anda muito por atualização.
- ✓ Convergência mais rápida.
- ✓ Maior risco de ultrapassar o vale ou oscilar em torno dele.



HIPERPARÂMETROS
OTIMIZADOR

O **otimizador** é o algoritmo responsável por **determinar como os pesos e biases serão atualizados durante o treinamento,** utilizando os **gradientes da função de perda calculados pela retropropagação (*backpropagation*).**

Os **gradientes** indicam como alterações em cada peso e *bias* afetam a função de perda e, portanto, fornecem a informação necessária para determinar **em que direção os parâmetros devem ser alterados para reduzir a perda.**

O **Gradiente Descendente (*Gradient Descent*)** constitui a ideia fundamental de otimização. Diferentes otimizadores utilizam os gradientes de maneiras distintas, buscando melhorar a **velocidade e a estabilidade da convergência.**



HIPERPARÂMETROS
OTIMIZADOR

Entre os principais estão:

- **SGD (*Stochastic Gradient Descent*)**
- **SGD com Momentum**
- **RMSprop**
- **Adam**
- **AdamW**

A taxa de aprendizado determina principalmente a escala/tamanho do passo; o otimizador determina a estratégia utilizada para realizar a atualização.